

HACKATHON CONCEPT NOTE

FAULT-TOLERANT REMOTE PATIENT MONITORING WITH PRIORITY ALARM RECOVERY

Problem 21 — Biomedical / Healthcare

BATCH NAME

FLAWLESS FEMMES

TEAM DETAILS

ROLE	NAME	ROLL NUMBER
Member 1	B. Bhavya Sri Lakshmi	23A91A0409
Member 2	A. Sri Harshitha	23A91A0402
Member 3	M. Rishitha Sravanthi	23A91A0440

Fault-Tolerant Remote Patient Monitoring with Priority Alarm Recovery

Problem 21 — Biomedical / Healthcare | Remote Patient Monitoring with Network Fault Recovery

Field	Details
Platform	QNX Momentics IDE and VM Ware Virtual Machine
Team size / duration	Three students / Two weeks
Core objective	Maintain patient monitoring through Wi-Fi/Ethernet outages and recover high-priority alarms without logical loss.

1. Problem Understanding

Remote patient monitoring must continue when the connection to a central station fails temporarily. During an outage, a patient node may still detect abnormal heart rate, low SpO₂, high temperature, or other urgent conditions. If monitoring depends on the network, critical events can be delayed or lost. The core challenge is therefore not only reconnecting; it is preserving clinically important information while keeping acquisition and alarm evaluation active.

Our system separates time-critical monitoring from unreliable communication. Each QNX patient node continuously samples vitalsign data, evaluates alarms locally, and stores unsent data in priority-aware buffers. Once connectivity returns, emergency alarms are replayed before routine telemetry, while sequence numbers and acknowledgments prevent duplicate alarms at the central station.

2. Proposed Solution

The prototype consists of one or more patient nodes and a central monitoring station. Each node uses simulated or available sensor inputs for heart rate, SpO₂, temperature, and blood pressure. The Patient Monitor produces timestamped records containing patient ID, sequence number, priority, payload, and checksum. The Alarm Task evaluates configurable thresholds and classifies records as emergency, warning, routine, or diagnostic.

The Buffer Manager stores records in separate bounded queues. Emergency alarms use reserved capacity so routine telemetry cannot evict them. During normal operation, the Network Task sends framed TCP records to the central station. The receiver validates the record, detects duplicate sequence numbers, logs valid data, and returns an application acknowledgment. A record is removed only after acknowledgment.

The Recovery Task uses heartbeats, timeout detection, socket-error handling, and exponential-backoff retry. On failure, the node enters offline-buffering mode but continues sampling and alarm detection. After a successful reconnect and handshake, it flushes emergency alarms first, then warning alarms, and finally routine records. A mandatory CLI exposes all operating states and test evidence.

3. Innovation and Originality

The proposed innovation is a priority-aware network recovery mechanism designed specifically for patient-monitoring data. Instead of replaying buffered records only according to arrival order, the system protects emergency alarms and transmits them before routine measurements.

The project also uses application-level reliability on top of TCP. Every record contains a sequence number and checksum, while the central station returns an explicit acknowledgment. If the connection fails after the receiver obtains a packet but before the sender receives the acknowledgment, the record may be retransmitted. The central station identifies it as a duplicate and acknowledges it without creating a second alarm.

This approach demonstrates reliable recovery, not just reconnection. The CLI will make the behavior visible by showing when the fault occurred, how long buffering lasted, which alarms were recovered, and whether duplicates or missing sequence numbers were detected.

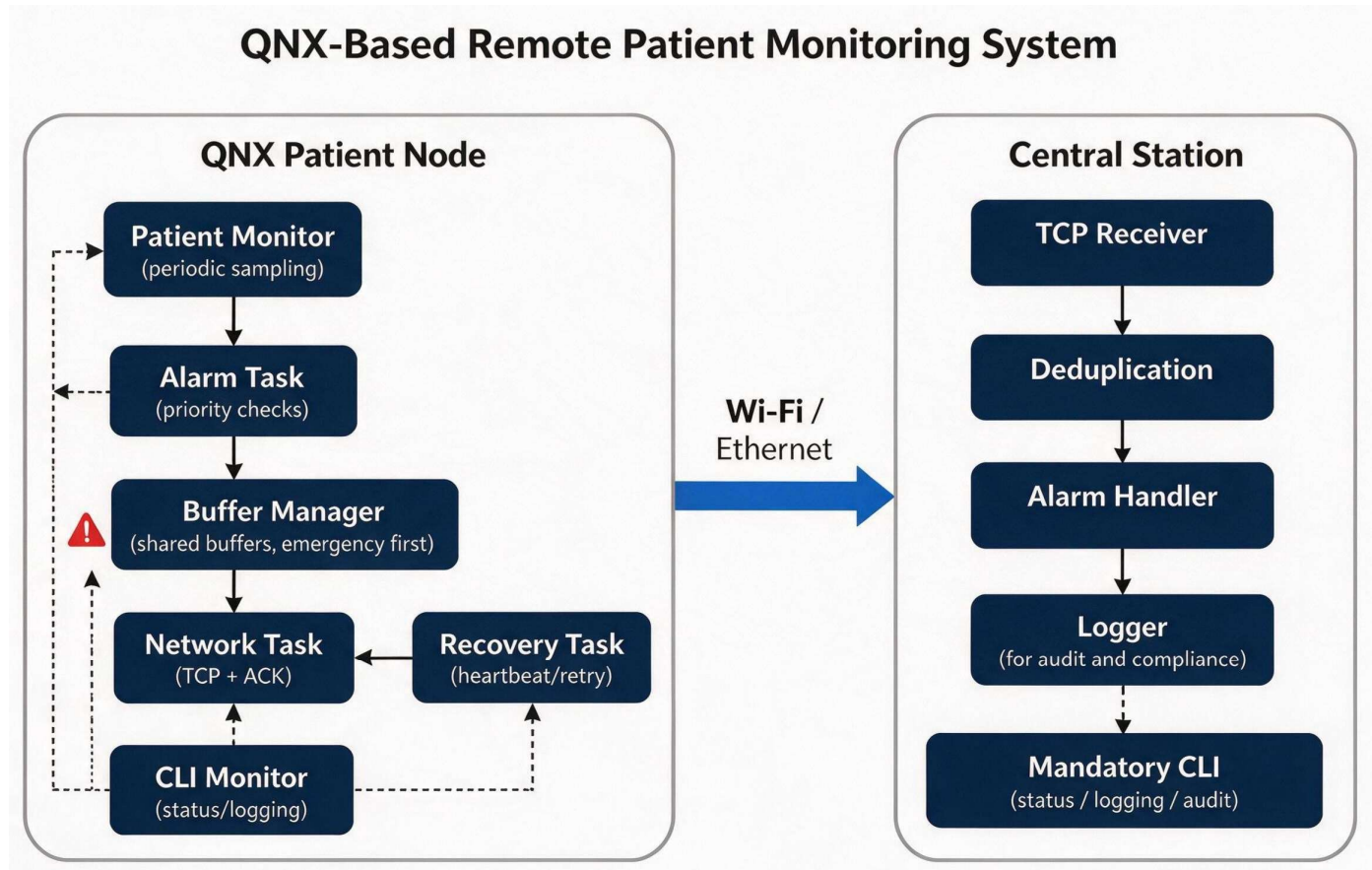
4. QNX/RTOS Design

QNX is necessary because the system has periodic acquisition, safety-relevant alarm deadlines, asynchronous network behavior, shared buffers, and fault recovery. The Patient Monitor must complete each sampling-and-queueing cycle before the next sampling release. The Alarm Task must not wait behind a stalled network transmission or noncritical CLI output. A priority-based RTOS design makes these timing relationships explicit and testable.

Process / Thread	Relative Priority	Reason
Alarm Task	Highest	Urgent threshold detection and emergency enqueueing must preempt noncritical work.
Patient Monitor	High	Periodic acquisition must meet its sampling period and remain independent of network state.
Buffer Manager	Medium-high	Moves records safely between producers and the transmitter; protects emergency capacity.
Network Task	Medium	Handles TCP send/ACK processing; may block without holding shared-buffer locks.
Recovery Task	Medium	Runs heartbeat, retry timers, reconnection, and priority-based flushing.
CLI / Logger	Low	Provides observability but must not affect safety-relevant execution.

QNX channels/messages and pulses carry control events such as ALARM_DETECTED, NETWORK_DOWN, NETWORK_UP, and FLUSH_BUFFER. Shared memory implements bounded ring buffers between local processes. The Recovery Task uses periodic timers for heartbeat checks and retry scheduling. The Network Task copies a bounded batch from the buffer before a blocking socket operation and releases the buffer lock before transmitting, thereby isolating acquisition and alarms from network delay.

5. Architecture



Data flows from the Patient Monitor to the Alarm Task, then to the priority buffer and Network Task. Control events flow through QNX IPC. The Recovery Task detects connection loss and controls reconnect-and-flush behavior. The central station validates incoming records, suppresses duplicates by sequence number, and provides a CLI view of node health, alarms, and recovery status.

6. Real-Time Targets

- Vital-sign sampling period: 100 ms for the prototype.
- Maximum sample-processing latency under normal load: ≤ 20 ms.
- Alarm detection and protected-buffer enqueue latency: ≤ 50 ms from sample creation.
- Maximum sampling jitter under normal load: ≤ 10 ms.
- Network-failure detection after missed heartbeat acknowledgments: ≤ 3 seconds.
- Reconnection attempt begins within 1 second after a retry timer expires.
- Emergency alarms are stored locally before transmission and flushed before routine records.
- CLI reports queue depth, recovery duration, retries, retransmissions, duplicates, and missing sequence numbers.

These are prototype targets to be measured during the hackathon, not claims of medical-device certification, zero latency, or absolute reliability.

7. Implementation Plan

- Configure QNX images, cross-compilation, Raspberry Pi networking, and central-station connectivity.
- Implement simulated vital-sign inputs, periodic acquisition, record format, timestamps, and CRC/checksum.
- Implement the central TCP receiver with framing, application acknowledgment, logging, and duplicate detection.
- Implement QNX processes, IPC events, shared-memory priority queues, and alarm thresholds.
- Implement heartbeat monitoring, retry backoff, reconnect handshake, and priority buffer flushing.
- Build the mandatory CLI and structured recovery-event log.
- Inject network failures, measure timing and recovery behavior, then refine priorities and buffer limits.

- Prepare final demo, test evidence, documentation, and optional dashboard only after the MVP is complete.

Team Member	Responsibility
Student 1 (23A91A0409)	QNX application design, processes/threads, priorities, IPC, shared-memory buffers.
Student 2 (23A91A0402)	Vital-input simulator, TCP protocol, heartbeat logic, reconnect and recovery flow.
Student 3 (23A91A0440)	CLI, event logging, test automation/fault injection, measurements, documentation.

8. Testing and Validation

Normal tests verify periodic sampling, threshold detection, correct packet framing, acknowledgment, and sequence ordering. Stress tests run multiple patient nodes, generate concurrent measurements and repeated alarms, and fill routine queues to confirm that emergency capacity remains protected. Fault tests disconnect Ethernet or Wi-Fi, stop/restart the central receiver, delay acknowledgments, corrupt packets, and interrupt a buffer flush.

We will measure sampling jitter, alarm latency, failure-detection time, retry count, recovery duration, maximum queue depth, missing sequence numbers, duplicate packets, and recovered alarms. The acceptance condition is that critical alarms remain in the protected buffer during the defined outage and are logically delivered after recovery. Routine measurements may be delayed, but they must not displace emergency events.

9. Expected Outcome and Impact

The final prototype will visibly demonstrate a QNX patient node continuing to monitor and evaluate vital signs during a communication outage. When connectivity returns, the central station will show emergency alarms delivered ahead of routine telemetry, with duplicate suppression and a complete recovery timeline. The solution is relevant to hospital wards, ambulances, home healthcare, remote clinics, and any distributed monitoring application where temporary network loss must not suppress urgent events.

10. Mandatory CLI Output

```
QNXMedLink Central Station
----- Patient
node      : patient-02
Network state : RECOVERING
Last heartbeat : 2.1 s ago
Buffer      : 184 records | Emergency: 2
Retry attempt : 4 | Duplicates: 1 | Missing: 0

Recovery timeline:
09:42:10 Network failure detected
09:42:11 Offline buffering started
09:42:18 Connection restored
09:42:18 Emergency alarm flush started
09:42:19 Emergency alarm flush completed
09:42:23 Routine traffic restored
```

11. Future Scope

Future work can add persistent storage for power-loss recovery, encrypted and authenticated communication, real medical sensors, time synchronization, configurable clinical thresholds, redundant central stations, and an optional Python/Qt/web dashboard. A production deployment would additionally require formal medical-device safety, cybersecurity, usability, and regulatory validation.